Software Requirements Specification (SRS)

Project: Chat Application (Socket Programming)
Version: 1.0
Authors: Team 8
Annanya Sharma
Aryan Shashank Musale
Kaushik Krishna Dwarakanath (Lead)
Kachhadiya Darshil Vipulkumar
Date: 7 September 2026
Status: Submitted / For Evaluation

Revision history

Version	Date	Author	Change summary	Approval
1.0	07-09-2026	Team 8	SRS for Chat Application; reformatted to prescribed template structure	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Chat Application developed by Team 8 as part of UE24CS341A – Software Engineering. It describes what the system shall do — enabling real-time, multi-user text communication using socket programming — and the constraints under which it shall operate. This document is intended to guide design, implementation and testing for all subsequent deliverables.

1.2 Scope

The Chat Application allows registered users to join a shared chat room and exchange text messages in real time using WebSockets. The system shows live typing indicators, tracks online/offline presence, and persists message history. It is a single-room, campus/community-group-oriented tool and is not intended to replace enterprise messaging platforms.

In Scope	Out of Scope
User registration and login	Multi-room / private one-to-one chat
Real-time message send/receive via sockets	Voice/video calling
Typing indicators	File/media sharing beyond text
Online/offline presence tracking	Push notifications on mobile
Message history persistence and retrieval	End-to-end encryption
Basic user profile management	Third-party identity provider login (Google/FB SSO)

1.3 Audience

- Development team (Team 8) — as the primary reference for design and implementation.
- Evaluators — to assess completeness and correctness of requirements.
- Testers — to derive test cases and acceptance criteria.

1.4 Definitions

- Socket — A persistent, bidirectional connection between client and server used for real-time data exchange.
- Room — A logical chat channel that groups a set of connected users who can message each other.
- Presence — The real-time online/offline status of a user.
- JWT — JSON Web Token, a signed token used to authenticate a user across REST and socket requests.
- Typing indicator — A transient UI signal shown to other users when someone is composing a message.
- Message history — The persisted, ordered log of past messages in a room, retrievable on room join.

2. Overall description

2.1 Product perspective

The Chat Application is a new, standalone, self-contained system. It consists of a browser-based frontend, a backend server that manages WebSocket connections and REST authentication endpoints, and a database for persisting users and messages. It does not integrate with any existing legacy system.

2.2 Major product functions (detailed)

- Register and authenticate users.
- Send and receive messages instantly across connected clients.
- Show typing indicators in real time.
- Track and display user presence (online/offline).
- Persist and retrieve message history.
- Manage basic user profile information.

2.3 User roles and characteristics (expanded)

User Class	Privileges	Characteristics
Registered User	Register, login, send/receive messages, view presence & history, edit own profile	Non-technical, casual/frequent use, expects instant feedback
Guest / Unauthenticated Visitor	View login/register screen only	First-time visitor

2.4 Operating environment

- Client: Any modern web browser (Chrome, Firefox, Edge) with WebSocket support.
- Server: Node.js runtime (Express + Socket.IO) or Python runtime (Flask-SocketIO), deployed on a single cloud/local host for the demo.
- Database: MongoDB (document store for users and messages).

2.5 Constraints

- Must be implementable within a 6-sprint academic timeline by a 4-member team.
- Limited to a single shared chat room for this version (per Deliverable 1 scope).
- Must use socket-based (not polling-based) real-time communication, per project brief.
- Users are assumed to have a stable internet connection during the chat session.
- The demo/test environment is expected to support at least 50 concurrent socket connections.
- Third-party libraries (Socket.IO, Express/Flask, MongoDB driver) are dependencies for implementation.

3. External interface requirements

3.1 User interfaces

- Login / Registration page — email, username, password fields with validation messages.
- Chat room page — message list, message input box, active user list, typing indicator strip.

3.2 Hardware interfaces

No dedicated hardware interface is specified for this version. Users interact with the application through standard client hardware such as a display, keyboard and pointing device supported by the web browser.

3.3 Software interfaces

Interface	Description
REST API	Handles registration, login and profile update requests (JSON over HTTPS).
Socket.IO API	Handles real-time events: connect, disconnect, sendMessage, receiveMessage, typing, stopTyping, presenceUpdate.
Database Driver	MongoDB driver (Mongoose for Node.js / PyMongo for Flask) for persisting users, messages and rooms.

3.4 Communications

- HTTPS for REST authentication endpoints.
- WebSocket (via Socket.IO protocol) for real-time bidirectional messaging.

4. System features (detailed)

Each requirement below includes priority, acceptance criteria and a reference verification method. IDs follow CHAT-FR-###.

4.1 User Authentication & Registration

Description: Allows a new user to create an account and an existing user to securely log in. Stimulus/Response: User submits registration or login form → system validates credentials and returns success/failure with a session token.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-001	The system shall allow a new user to register using a unique username, email and password.	Functional	High	Development / Security	Successful registration creates a new user account.	Auth Service
CHAT-FR-002	The system shall allow a registered user to log in using email/username and password.	Functional	High	Development / Security	Valid credentials produce an authenticated session.	Auth Service
CHAT-FR-003	The system shall hash user passwords before storing them in the database.	Functional	High	Development / Security	Stored user records contain a password hash rather than plaintext password.	Auth Service
CHAT-FR-004	The system shall issue a session token (JWT) on successful login and validate it on each socket connection.	Functional	High	Development / Security	A valid JWT is accepted and an invalid/expired token is rejected.	Auth Service / Socket Gateway

4.2 Real-time Messaging

Description: Core feature enabling instant message exchange between connected users via WebSockets.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-005	The system shall establish a persistent WebSocket connection between an authenticated client and the server.	Functional	High	Development / Messaging	An authenticated client establishes a socket connection successfully.	Socket Gateway

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-006	The system shall broadcast a sent message to all users currently connected to the chat room.	Functional	High	Development / Messaging	A sent message is received by all connected users in the room.	Message Service / Chat UI
CHAT-FR-007	The system shall deliver messages to connected clients within 1 second under normal load.	Functional	High	Development / Messaging	Messages meet the 1-second delivery target under normal load.	Socket Gateway

4.3 User Presence & Status

Description: Tracks and displays which users are currently online. Stimulus/Response: Client socket connects/disconnects → server updates and broadcasts presence state.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-008	The system shall mark a user as “online” when their socket connects and “offline” when it disconnects.	Functional	High	Development / Presence	Presence changes are reflected after connection and disconnection.	Presence Manager
CHAT-FR-014	The system shall display the current active user list for connected users.	Functional	High	Development / Presence	Connected users are shown in the active user list.	Presence Module / User List UI

4.4 Typing Indicators & Message History

Description: Shows live typing feedback and persists/retrieves past messages. Stimulus/Response: User starts typing → typing event broadcast; user joins room → recent messages fetched from the database.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-009	The system shall emit a “typing” event to other users when a user begins typing.	Functional	Medium	Development / Typing	Other connected users receive the typing event.	Typing Service
CHAT-FR-010	The system shall store all sent messages in the database with sender ID, room ID and timestamp.	Functional	High	Development / History	Stored messages contain sender, room and timestamp fields.	History Service
CHAT-FR-012	The system shall display and clear a typing indicator for other users in the room.	Functional	Medium	Development / Typing	Typing indicator is shown while a user is typing and cleared when typing stops/expires.	Typing Service / Chat UI
CHAT-FR-013	The system shall retrieve the last 50 messages when a user joins the chat room.	Functional	High	Development / History	The latest 50 stored messages are displayed after room join.	History API / Chat UI

4.5 Chat Room & Profile Management

Description: Supports joining the shared room and managing basic user profile information.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-FR-011	The system shall allow an authenticated user to join the shared chat room.	Functional	High	Development / Room	A logged-in user is added to the active user list after joining.	Room Service / Chat UI
CHAT-FR-015	The system shall allow an authenticated user to update basic profile information.	Functional	Medium	Development / Profile	Submitted profile changes are validated and persisted.	Profile Service / REST API

5. Non-functional requirements (detailed)

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
CHAT-NFR-001	95% of messages shall be delivered to all connected clients within 1 second under a 50-concurrent-user test load.	Performance	High	95th percentile delivery time ≤ 1 s with 50 concurrent users. Test: TC-PERF-01
CHAT-NFR-002	The chat server shall be available at least 98% during the defined operating/demo window, excluding planned maintenance.	Availability	High	Uptime ≥ 98% in the operating/demo window (operational review)
CHAT-NFR-003	A new user shall be able to register and send a first message in no more than 3 screens.	Usability	Medium	Task-based walkthrough completed in ≤ 3 screens. Test: TC-UX-01
CHAT-NFR-004	A dropped WebSocket connection shall auto-reconnect within 5 seconds without message loss.	Reliability	High	Reconnect ≤ 5 s with no lost messages in network-drop scenarios. Test: TC-REL-01
CHAT-NFR-005	Socket event handlers, authentication logic and database access shall be separated into identifiable modules with unit tests.	Maintainability	Medium	Code review confirms module separation; unit tests exist for each module. Test: TC-MAINT-01

5.1 Security

5.1.1 Security Objectives

- Protect user credentials and session authentication so that only authenticated users can access chat functions.
- Protect chat content and application inputs from unauthorized access and unsafe client-side rendering.

5.1.2 Security Requirements

Req ID	Requirement	Type	Priority	Acceptance criteria / Test case ref
SEC-001	Passwords shall not be stored in plaintext.	Security	High	TC-SEC-01

Req ID	Requirement	Type	Priority	Acceptance criteria / Test case ref
SEC-002	Socket connections shall be authenticated using a valid session token before joining a room.	Security	High	TC-SEC-02
SEC-003	The system shall validate and sanitize all user-supplied message content before broadcast and storage.	Security	High	TC-SEC-03
SEC-004	Expired or invalid session tokens shall be rejected on both REST and socket endpoints.	Security	High	TC-SEC-04
SEC-005	Cross-site scripting (XSS) payloads in chat messages shall be escaped before rendering on the client.	Security	High	TC-SEC-05
SEC-006	Failed login attempts and unauthorized socket connection attempts shall be logged.	Security	Medium	TC-SEC-06

6. Quality attributes & Acceptance tests

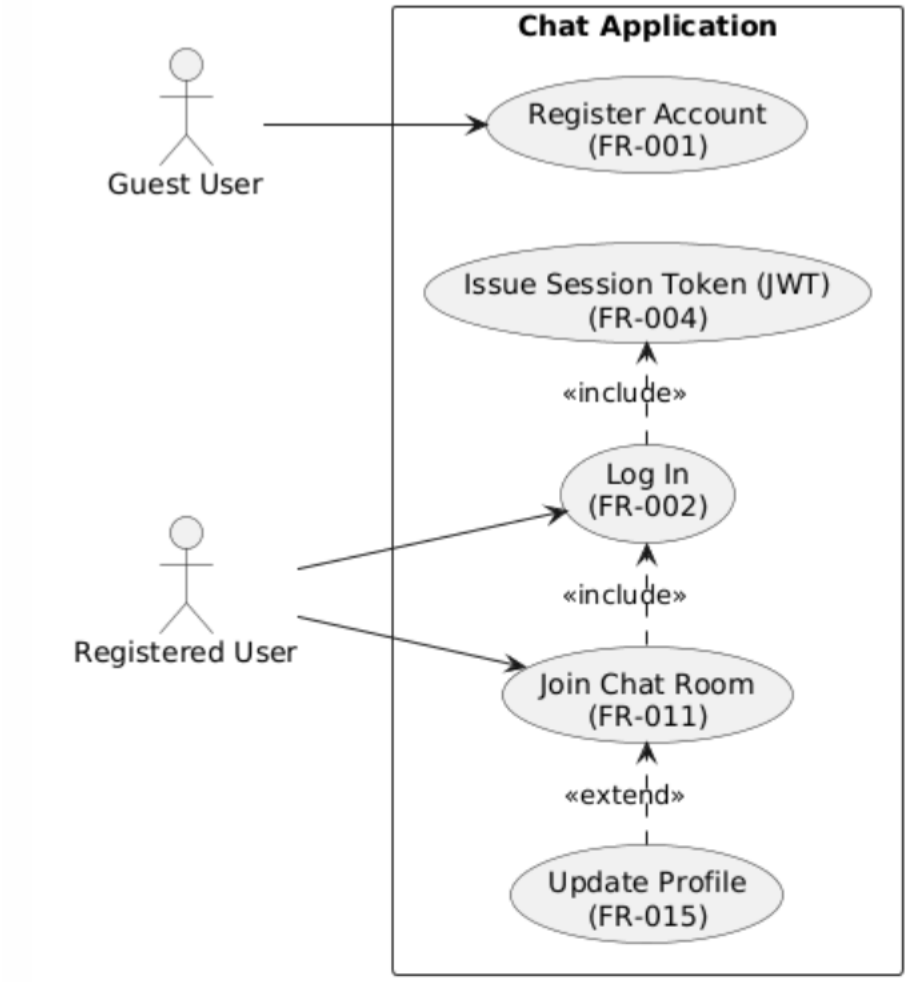
The acceptance approach is based on the validation specification in the current SRS. Acceptance requires the applicable requirement tests to pass without critical failures.

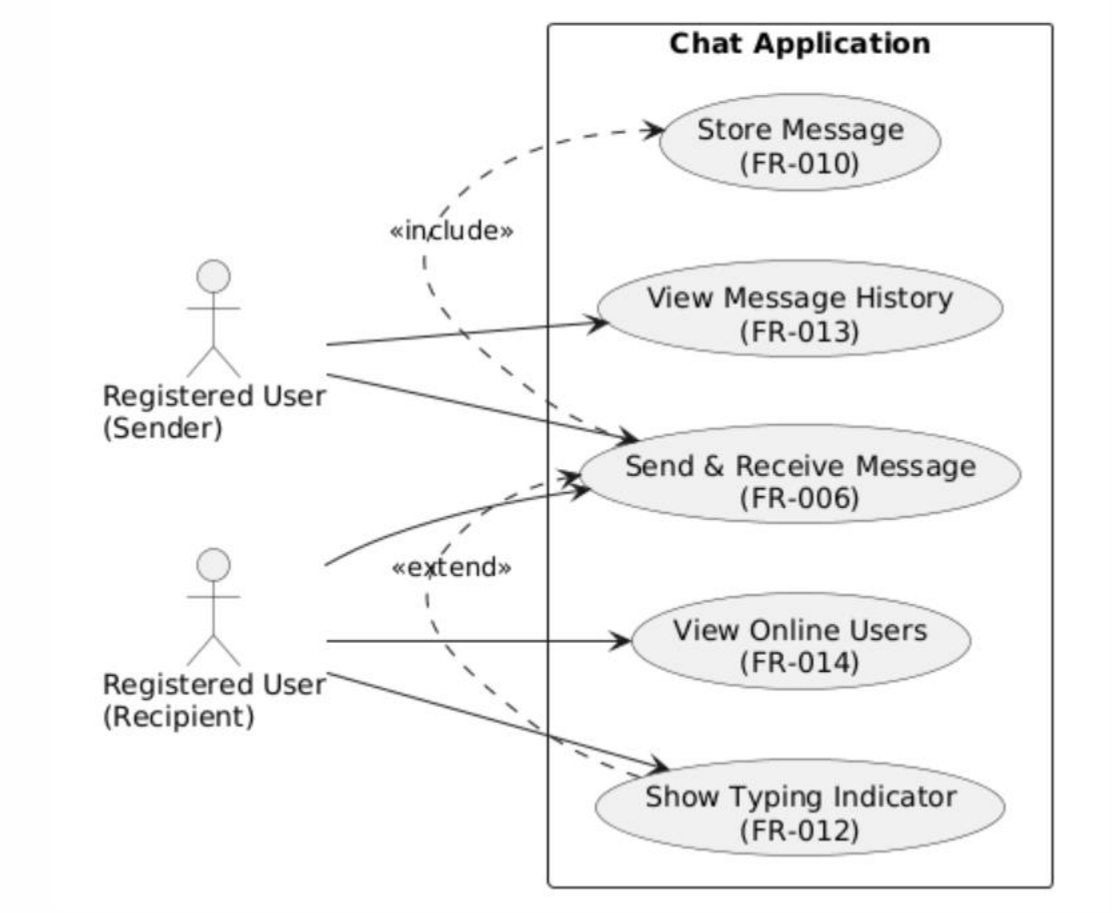
Quality attribute	Requirement / Target	Acceptance test
Performance	95% of messages delivered within 1 second with 50 concurrent users.	Load/response-time test
Availability	At least 98% during the defined operating/demo window, excluding planned maintenance.	Operational review
Usability	New user registers and sends a first message in no more than 3 screens.	Task-based walkthrough
Reliability	Dropped socket reconnects within 5 seconds without message loss.	Failure/reconnect scenarios
Maintainability	Core handlers, authentication and DB access are separated into identifiable modules with unit tests.	Automated unit tests + code review
Security	Authentication, authorization, input validation and XSS protections are verified.	Security testing

6.1 Acceptance test suites

- Functional: black-box functional tests covering valid/invalid login and message send.
- Security: authentication/authorization and security testing, including unauthenticated socket connection attempts.
- Performance: load/response-time test with 50 concurrent users sending messages.
- Usability: task-based walkthrough where a new user registers and sends the first message.
- Reliability: network-drop and auto-reconnect scenarios.
- Unit: automated unit tests and coverage for message validation and password hashing.

7. UML Use-Case Diagram


Chat Application
Guest User
Register Account
(FR-001)
Issue Session Token (JWT)
(FR-004)
«include»
Log In
(FR-002)
Registered User
«include»
Join Chat Room
(FR-011)
«extend»
Update Profile
(FR-015)



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
FR-001	User registration	4.1 / DS-Auth-01	REST Auth API, MongoDB	TC-AUTH-01, TC-AUTH-02	N	
FR-002	User login	4.1 / DS-Auth-01	REST Auth API	TC-AUTH-03, TC-AUTH-04	N	
FR-003	Password hashing	4.1 / DS-Auth-01	REST Auth API	TC-AUTH-06	N	
FR-004	JWT issue and socket validation	4.1 / DS-Auth-02	REST Auth API, Socket Gateway	TC-AUTH-05, TC-SEC-02	N	
FR-005	Persistent WebSocket connection	4.2 / DS-Sock-01	Socket Gateway	TC-MSG-01	N	

FR-006	Broadcast message to room	4.2 / DS-Msg-01	Socket Gateway, Message + History Service	TC-MSG-02	N	
FR-007	Delivery within 1 second	4.2 / DS-Sock-01	Socket Gateway	TC-PERF-01	N	
FR-008	Online/offline presence	4.3 / DS-Pres-01	Presence + Room Service	TC-PRES-01, TC-PRES-02	N	
FR-014	Active user list	4.3 / DS-Pres-01	Presence + Room Service, Browser Client	TC-PRES-01, TC-PRES-02	N	
FR-009	Emit typing event	4.4 / DS-Type-01	Socket Gateway (typing handler)	TC-TYPE-01	N	
FR-010	Store messages (sender, room, timestamp)	4.4 / DS-Hist-01	Message + History Service, MongoDB	TC-HIST-01, TC-HIST-02	N	
FR-012	Show and clear typing indicator	4.4 / DS-Type-01	Browser Client, Socket Gateway (typing handler)	TC-TYPE-01	N	
FR-013	Retrieve last 50 messages	4.4 / DS-Hist-01	Message + History Service, MongoDB	TC-HIST-01, TC-HIST-02	N	
FR-011	Join shared room	4.5 / DS-Room-01	Presence + Room Service	TC-ROOM-01	N	
FR-015	Profile update	4.5 / DS-Prof-01	Profile Service, REST API	TC-PROF-01	N	
NFR-001	95% delivered within 1 s at 50 users	5 / DS-Perf-01	Socket Gateway	TC-PERF-01	N	
NFR-002	98% availability in demo window	5 / DS-Avail-01	Application server	TC-AVAIL-01	N	
NFR-003	Register and send first message in ≤ 3 screens	5 / DS-UX-01	Browser Client	TC-UX-01	N	
NFR-004	Auto-reconnect within 5 s without message loss	5 / DS-Rel-01	Socket Gateway, Browser Client	TC-REL-01	N	
NFR-005	Modular code with unit tests	5 / DS-Maint-01	All modules	TC-MAINT-01	N	

SEC-001	No plaintext passwords	5.1.2 / DS-Sec-01	REST Auth API	TC-SEC-01	N	
SEC-002	Socket authentication before joining room	5.1.2 / DS-Sec-02	Socket Gateway	TC-SEC-02	N	
SEC-003	Validate and sanitize message content	5.1.2 / DS-Sec-03	Message + History Service	TC-SEC-03	N	
SEC-004	Reject expired/invalid tokens (REST and socket)	5.1.2 / DS-Sec-04	REST Auth API, Socket Gateway	TC-SEC-04	N	
SEC-005	Escape XSS payloads on render	5.1.2 / DS-Sec-05	Browser Client	TC-SEC-05	N	
SEC-006	Log failed logins and unauthorized sockets	5.1.2 / DS-Sec-06	REST Auth API, Socket Gateway	TC-SEC-06	N	